

エディタとスクリプト

基礎生物学研究所

GITC 2026 春インフォマティクス入門

杉浦宏樹

作業場所

- 以降の作業は、以下のディレクトリで行います。

```
~/gitc/data/6_editor/
```

cd コマンドを用いてディレクトリを移動し、

pwd コマンドを利用して、カレントディレクトリが上記になっていることを確認してください。

(Tabキーの補完機能を積極的に活用しましょう)

テキストエディタ

テキストエディタ

- 中身が文字だけのファイルを編集するときに使うソフトウェア
- Windowsの「メモ帳」や、MacOSの「テキストエディット」もテキストエディタの一つ
 - …キーボードやマウスを用いて操作
- リモートログイン先等、マウスが使えないこともある。
その場合、キーボードのみでも便利に使用できるのが
Unixでも使用できるテキストエディタ

Unixで使える代表的なテキストエディタ

- **emacs**

- 「Control+文字キー」または「Esc+文字キー」によってカーソル移動等の操作を行う
- 「Esc x コマンド」によって、豊富なコマンドを利用可能

- **vi (vim)**

- 文字入力モードとコマンド入力モードの2つのモードを切り替えて使う
 - 以下"入力モード"と"コマンドモード"と記載
- Unixでは最も標準的なエディタ

- **本講義ではvi(vim)を使用します**

Vi(Vim)とは

- Viとは、UNIX/Linux 系OSで標準的に使われてきたキーボード操作を中心とした歴史のあるテキストエディタ。
- 現在使われている Vi は、多くの場合 Vim(Vi Improved)という拡張版ですが、基本的な操作方法や考え方は共通しています。
そのため、本資料ではまとめて「**Vi**」と呼びます。
- Viの最大の特徴は、「2つの操作モード」を切り替えて使う点
 - 「コマンドモード」 : 移動・削除・コピーなどの操作を行う
 - 「入力モード」 : 文字を入力する

2つの操作モードの違いを理解することがVi操作の最大の壁

Vimを扱う際の注意

- マウスは使えません。

メモ帳やテキストエディットとは異なり、
マウスを使用しようとしてもできません。
(カーソル移動や範囲選択などもできません)

→代わりにキーボードを使用します。

2つの操作モードについて

1. コマンドモード

- 起動時の初期状態。**文字は入力できません。**
- **カーソル移動、削除、コピー、保存、終了**などの「コマンド(命令)」を実行するモード

2. 入力モード

- **文字を自由に入力するためだけ**のモード。
- キーボードで文章を入力でき、編集できる状態。

操作について迷ってしまったら、**Escキー**を押す！

1回のEscキー入力でノーマルモードに戻れます

不安なら何回Escキーを押しても大丈夫！

2つの操作モード - コマンドモード -

- 目的: ファイルに対する指示(コマンド)を実行
- 移行: 初期状態。あるいは編集モード時にEscキーを押す。
- 主な操作(コマンド)

操作の種類	コマンド	内容
カーソル移動	h j k l (← ↓ ↑ →)	左/下/上/右 矢印キーでも操作できますが、慣れるとhjklの方が楽
削除	x	カーソル位置の1文字を削除 (ノーマルモード時はバックスペースでは削除できない)
削除	dd	カーソル行を削除
アンドゥ	u	直前の操作を元に戻す
保存	:w	ファイルを保存
終了	:q	Viを終了

保存・終了は
後ほど詳しく解説

2つの操作モード－入力モード－

- 目的
 - ファイルに文字を入力する
- 移行
 - コマンドモードの状態から、以下のキーを押す。
 - **i**: Insert (まずはこれを覚える！)
 - カーソル位置から文字入力を開始
 - **a**: Append
 - カーソル位置の次から文字入力を開始
 - **o**: Open
 - カーソル行の下に新しい行を追加し、文字入力を開始
- 解除
 - Escキーを押すことで、コマンドモードに戻る

基本操作について – 起動・編集・終了

1. 起動

- ターミナル上で **vi samtools_readme.txt** と入力
 - samtools_readme.txt が存在しない場合、新しくファイルが作成されます。
 - すでに samtools_readme.txt がある場合は、その内容を開いて編集します。(操作方法は同じ)

2. 編集

- 初めはコマンドモードであるため、**i** を押して入力モードへ移行
- 編集モード中に自由に文字を入力してください。
- 文字の入力が終わったら **Escキー** を押してコマンドモードへ移行

3. 終了(保存)

- コマンドモードで **:** (コロン) から始まるいずれかのコマンドを入力する

コマンド	意味	使う場面
:wq	保存して終了	編集内容を保存してviを終了したいとき
:w 別名	別名で保存	新しい名前で保存したいとき
:q!	保存せずに強制終了	保存したくないとき/今すぐviを終了したいとき

よくあるトラブルと対処法

- viの操作で初心者がつまずくのは、「モード」が分からなくなること。迷った時は、「コマンドモードに戻る」習慣をつけましょう。
 - コマンドモード: キーを押しても文字が出ず、コマンドとして認識
 - 入力モード: キーを押すとそのまま文字が入力される

状態(トラブル)	原因	対処法	内容
文字が入力できない	コマンドモードにいる	i キーを押す	入力モードに移行し、文字を打ち始められます。
画面にコマンド文字が出た/勝手に消えた	コマンドモードで、意図せずキーを押してしまった	Esc キーを押す	実行中の操作を止め、コマンドモードに戻ります。
変な文字が出た	誤って特殊キーを押してしまった	Esc キー → u キー	まずコマンドモードに戻り、u(アンドゥ)で直前の操作を取り消す。
とにかく何が起きたか分からない		Escキーを2-3回程度押す	「迷ったら Esc」 Escキーは何度押しても問題なく、確実にコマンドモードに戻れます。

編集効率を上げる便利コマンド (コマンドモード)

コマンド	意味	使う場面
w	次の単語の先頭に移動	長い行の中で数単語先にカーソルを合わせるとき
b	前の単語の先頭に移動	長い行の中で数単語前にカーソルを合わせるとき
0 (ゼロ)	行の先頭に移動	行の最初から編集・確認したいとき
\$ (ドル)	行の末尾に移動	行の最後を確認・追記したいとき
gg	ファイルの先頭に移動	ファイルの最初から見直したいとき
G	ファイルの末尾に移動	ファイルの最後一気に移動したいとき
/ (スラッシュ)	文字列検索 (例: /text)	特定の単語やコードの場所に移動したいとき
yy	カーソルがある行をコピー	行を丸ごと複製したいとき
p	ペースト(貼り付け)	コピーした行を下に貼り付けたいとき

基本操作について – 起動・編集・終了

- 終了(保存)
 - コマンドモードで:(コロン)から始まるいずれかのコマンドを入力する

コマンド	意味	使う場面
:wq	保存して終了	編集内容を保存してviを終了したいとき
:w 別名	別名で保存	新しい名前で作成したいとき
:q!	保存せずに強制終了	保存したくないとき/今すぐviを終了したいとき

操作がわからなくなった際は

Escキーを押してから
:q!

を入力して保存せずに強制終了

シェルスクリプト

シェルスクリプトとは？

- シェルスクリプトとは、あらかじめ実行したい一連のコマンドを記述したテキストファイルである。
- 実行権を持たせることによってシェルから実行できる。
- 実行するコマンド群を記録しておいて再利用することができる。
- 制御文を用いることで簡易的なプログラムとして実行可能。

シバン (shebang)

- シェルスクリプトのファイルには、先頭の行に下記のような記述が見られる。

```
#!/bin/sh
```

「#!」から始まる行を
シバンと呼ぶ

- シバンで指定されるパスにより、
2行目以降の文字群を実行すべきプログラム
(インタプリタ) を指定する。

```
例) rubyの場合  
#!/usr/bin/ruby
```

シェルスクリプト例

- 実際にシェルスクリプトを見てみよう
(以下、シバンは省略します)

```
$ less example1.sh
```

と入力し、example1.shの中を確認。

example1.sh

```
grep 'exon' human_chr1.gtf > line.tmp  
echo 'The number of exons:'  
wc -l line.tmp  
rm line.tmp
```

- 上記コマンドを一つずつ確認

復習：コマンド入力

- 以下のコマンドを順番に入力してください。

```
grep 'exon' human_chr1.gtf > line.tmp
```

```
echo 'The number of exons:'
```

```
wc -l line.tmp
```

```
rm line.tmp
```

シェルスクリプト実行例

example1.sh

```
grep 'exon' human_chr1.gtf > line.tmp  
echo 'The number of exons:'  
wc -l line.tmp  
rm line.tmp
```

演習)

\$./example1.sh

と入力し、このシェルスクリプトを実行せよ。

変数

- **変数**とは、値の出し入れができる箱のようなもの
- 変数は「数値」や「文字列」を値としてとる
- 変数を使うことでプログラムの動作を柔軟に変更可能
- 変数を使う前に値を代入する必要がある

```
name='yamada'
```

変数 name に 'yamada' という文字列（値）を代入する。
変数、イコール、入れる値の間にスペースは入れない。

```
echo ${name}
```

変数の中の値を呼び出すには、\$を使う。
\$のあとに続く文字列が変数であることを示すため、
{ } を使用して区別するとよい。

クォーテーションの違い

- シングルクォーテーション ['']

たとえ中に変数があったとしても中身を展開せず単に文字列として扱われる。

シェルに解釈されたくない場合に有効。

- ダブルクォーテーション ["]

中身に変数がある場合、シェルはその中の値を採用する。

```
name='yamada'  
echo '${name}' // ${name} と表示される。  
echo "${name}" // yamada と表示される。
```

変数の使用

- 変数を使用したシェルスクリプトの例を見てみよう。

example2.sh

```
param='exon'  
grep "${param}" human_chr1.gtf > line.tmp  
echo "The number of ${param}s:"  
wc -l line.tmp  
rm line.tmp
```

演習)

\$./example2.sh

と入力し、このシェルスクリプトを実行せよ。

変数の変更

- 先程の example2.sh では、“exon”という文字列でしか検索できない。他の文字列を検索したい場合、変数の中身を変えれば良い。

example3.sh

```
param='start_codon'  
grep "${param}" human_chr1.gtf > line.tmp  
echo "The number of ${param}s:"  
wc -l line.tmp  
rm line.tmp
```

演習)

example2.sh をVim で開き、上のようにparam= の値をstart_codonに編集した後、example3.sh という別名で保存せよ。

\$./example3.sh を実行すると何が起こるか？

スクリプトの実行権

先ほどviで作成した example3.sh を実行してみよう。

```
$ ./example3.sh
```

“Permission denied” と出たでしょうか？

ファイルに**実行権**がない場合、このようなエラーメッセージが出る。
chmodコマンドを使用して、**実行権**を与えよう。

```
$ chmod u+x example3.sh  
$ ./example3.sh
```

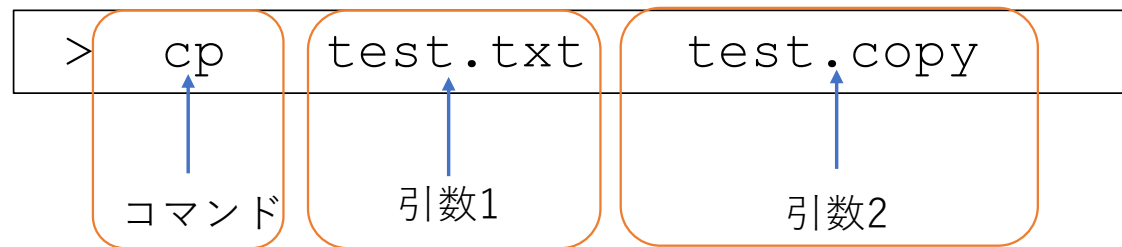
実行結果はどうなるか？

(参考) 変数と引数

- 先ほどの例では、**変数**に入れる値としてexonという文字列を使用した。
- しかし、別の文字列を使用したい場合、毎回スクリプトを直さないといけない。
- そのような手間を省くために、**引数**を使用することもできる。

(演習) 引数

- **引数**とは、コマンド実行時にコマンドラインから渡される値である。



- シェルスクリプトの中でも引数を利用できる。
その場合、スクリプト内で\$1などと表記する。

\$1	1 番目の引数	\$2	2 番めの引数	...	\$9	9 番目の引数
\$0	コマンド自身	\$*	引数全部			(などなど)

(演習) 引数の使用

- 検索に使用する文字列を引数として使用するシェルスクリプトを見てみよう。

example4.sh

```
#!/bin/sh
param=$1
grep "${param}" human_chr1.gtf > line.tmp
echo "The number of ${param}s:"
wc -l line.tmp
```

このスクリプトは、引数を1つとる。引数は変数 param に代入される。
このスクリプトを実行するためには、下記のような文法で入力を行う。

```
$ ./example4.sh exon
```

条件分岐と繰り返し
(if文、for文)

条件分岐と繰り返しとは？

- シェルスクリプトは、人が行っていた作業を自動化するためのもの
- 毎回同じ操作を手で行うのは非効率で、ミスも起きやすい
- if文:
 - 「もし～なら、～する」という条件による判断をスクリプトに書ける
- for文:
 - 同じ処理を、複数の対象に対して繰り返し実行できる
- if文・for文を使うことで、状況に応じた処理やまとめた処理が可能

条件分岐と繰り返しの強み

- ディレクトリにある各1000個のPDFや画像、音声データを仕分けたい
 - 手動での仕分けの場合
 - 目視での判断
 - 大量のドラッグ&ドロップ
 - 単調でつまらない動作で判断ミス多発
 - スクリプトでの仕分けの場合
 - for文
同じ処理を複数の対象に繰り返し実行
 - 全てのファイルを上から順に取る
 - if文
もし～なら、～する
 - もしPDFならこのディレクトリに移動
 - そうでない場合は別のディレクトリに移動
 - ルール通りの仕分けで判断ミスがない



if文とは？

- if文は、条件が成り立つときだけ処理を実行するための仕組み
 - これにより状況に応じて処理の流れを変えることができる
- ファイルの有無や値の一致などをスクリプトが自動判断可能
 - その結果、不要な処理やエラーを事前に防ぐことが可能になる
- 人が行っていた「ある／ない」「合っている／違う」という判断を、そのままスクリプトに書けるようになる
- if文は「判断」を自動化する仕組み

if文の基本構文

- if文の基本構文は次のとおりです。

```
if [ 条件 ]; then
    実行したい処理
fi
```

- if で条件判定を開始
 - 条件を満たさない場合は**何も処理を行わない**。
- [] の条件の前後には必ずスペースが必要
 - [条件] はNG、**[条件]**が正解
- then の下に、条件が成り立ったときの処理を書く
 - 必須ではないが、実行したい処理の前にインデント(空白文字)を入れて見やすくするのが一般的。
 - 空白文字を何文字入れるかの決まりはないが、全角スペースを入れてはいけない。
 - 基本的にはTabキーを1回押すのが一般的(スペースキーを入力しなくても自動的にインデントを入れてくれる)
- fi でif文の終わりを示す
 - then と fi はセットで必須。

if文の基本構文

- if文の基本構文に、**else** を追加できる
- else を使うと、「もし～ならA、そうでなければB」という条件分岐を書けるため
条件が成り立たなかった場合の処理を指定できる

```
if [ 条件 ]; then
    条件がOKだったときの処理
else
    条件がNGだったときの処理
fi
```

- else がない場合
→ 条件がNGだったときは**何も実行されない**
- else を加えると
→ **どちらか一方の処理が必ず実行される**

よく使う条件の書き方

- if文では、さまざまな「条件」を指定できる
ここでは 特によく使う基本的な条件 を紹介する
- ファイル・ディレクトリの判定
 - [-f file.txt] : file.txt というファイルが存在する場合に真
 - [-d dir] : dir というディレクトリが存在する場合に真
- 文字列の比較
 - ["\$a" = "\$b"] : 2つの文字列("\$a"と"\$b") が等しい場合に真
 - ["\$a" != "\$b"] : 2つの文字列("\$a"と"\$b") が異なる場合に真
 - どちらの場合も、**変数はダブルクォーテーションで囲む習慣をつけましょう**
- 数値の比較（基本）
 - ["\$n" -eq 10] : "\$n"が数字の10と等しい場合に真
 - ["\$n" -gt 5] : "\$n"が数字の5よりも大きい場合に真 (5を含まない)
 - ["\$n" -lt 5] : "\$n"が数字の5よりも小さい場合に真 (5を含まない)

条件を使ったif文の例1

- ファイルが存在するかを判定する

if_file.sh

```
if [ -f sample.txt ]; then
    echo "sample.txt は存在します"
else
    echo "sample.txt は存在しません"
fi
```

- sample.txt が 存在する場合
 - 「存在します」と表示される
- sample.txt が 存在しない場合
 - 「存在しません」と表示される
- 条件によって、実行される処理が自動で切り替わる

条件を使ったif文の例2

- 引数を使った文字列比較

if_arg.sh

```
if [ "$1" = "yamada" ]; then
    echo "引数は yamada です"
else
    echo "引数は yamada ではありません"
fi
```

(実行例) ./sample_if.sh yamada

よくある間違い
クォートなし (NG)
[\$1 = "yamada"]
スペースなし (NG)
["\$1"="yamada"]

条件式[]の内部には
クォートとスペースが必要

- 引数が yamada の場合
 - 「引数は yamada です」と表示される
- 引数が yamada ではない場合
 - 「引数は yamada ではありません」と表示される
- 今回は引数を比較したが、同様の書き方で変数を使った文字列比較も可能

for文とは？

- for文は、同じ処理を複数回くり返し実行するための仕組み
 - これにより、1つずつ手作業で行っていた処理をまとめて実行できる
- 複数のファイルや値を、順番に自動で処理することが可能
 - その結果、同じ操作を何度も書く必要がなくなり、作業量を減らせる
- 人が行っていた「これを順に全部やる」という作業を、そのままスクリプトに書けるようになる
- for文は「繰り返し」を自動化する仕組み

for文の基本構文

- for文の基本構文は次のとおりです。

```
for 変数 in 値の一覧; do  
    実行したい処理  
done
```

- for で繰り返し処理を開始
 - 指定した回数・対象に対して、同じ処理を順番に実行する
- for 変数 in 値の一覧 が for文の中心となる部分
 - 「一覧の中の値を、1つずつ取り出す」という意味
- 繰り返しのたびに、一覧の値が変数に代入される
 - 変数の中身が順番に入れ替わりながら処理が実行される
- 一覧には、複数の値やファイル名などを指定できる
 - スペース区切りで並べたものが「値の一覧」になる

for文の例1

- 数値の一覧を順番に処理する
for_num.sh

```
for n in 1 2 3; do  
    echo $n  
done
```

- **1 2 3** が「値の一覧」
 - この一覧の中身を 上から順に 処理する
- **n** が、一覧の値を一時的に入れる変数
 - 繰り返しごとに中身が変わる
- 実際に行われる処理
 - 1回目：n=1 → echo 1
 - 2回目：n=2 → echo 2
 - 3回目：n=3 → echo 3
- 値の一覧から1つずつ取り出し、同じ処理を順番に実行する

- for文の基本構文

```
for 変数 in 値の一覧; do  
    実行したい処理  
done
```


for文の例2

- 複数のファイルを順番に処理する
for_file.sh

```
for f in a.txt b.txt c.txt; do
    wc "$f"
done
```

- a.txt b.txt c.txt** が「処理したいファイルの一覧」
 - 一覧の数だけ、同じ処理が自動で繰り返される
- f** が、1つずつファイル名を受け取る変数
- 実際に行われる処理
 - 1回目：f=a.txt → wc a.txt
 - 2回目：f=b.txt → wc b.txt
 - 3回目：f=c.txt → wc c.txt
- 複数のファイルに対して wc コマンドを一括で実行する

- for文の基本構文

```
for 変数 in 値の一覧; do
    実行したい処理
done
```

for文とif文を組み合わせた例

- 複数のファイルに対して「存在するものだけ処理する」

for_if.sh

```
for f in a.txt b.txt c.txt x.txt; do
    if [ -f "$f" ]; then
        wc "$f"
    else
        echo "$f は存在しません"
    fi
done
```

- a.txt b.txt c.txt が処理したいファイルの一覧
- for 文で、ファイル名を1つずつ f に代入
- if 文で、そのファイルが存在するかを判定
 - 存在する → wc を実行 / 存在しない → メッセージを表示
- この判断を、ファイルの数だけ自動で繰り返す

if文・for文まとめ

- if文：条件によって処理を分ける仕組み
 - 条件によって処理を分ける仕組み
 - ファイルの有無や値の一致などを自動で判断できる
 - 「もし～なら、～する」という人の判断をスクリプトに書ける
- for文：同じ処理を繰り返し実行する仕組み
 - 同じ処理を複数回まとめて実行する仕組み
 - 数値やファイル名など、複数の対象を順番に処理できる
 - 手作業で行っていた繰り返し作業を一括で自動化できる
- if × for の組み合わせ
 - 複数の対象に対して、条件を確認しながら処理できる
 - 「存在するファイルだけ処理する」など、実用的なスクリプトが書ける

確認・判断・繰り返しを含む作業を自動処理するためのツールとして
if文、for文を活用してください

補足資料(emacsについて)

今回はviについて解説を行いました、emacsについての解説も記載します。
講義では使用しませんが、viが扱いにくいと感じた場合はemacsもお試してください。
お時間がある時にこちらをご確認ください。

emacsの導入について

emacsというソフトウェアを使用する場合はインストールが必要になる場合があります。

- 受講生の方

RCCS上ではインストールせずにemacsを利用可能です。

- 聴講生の方

mac(ターミナル)、windows(WSL)ともにemacsのインストールが必要です。

本講で扱うコマンド

- `$ emacs` Emacsエディタの起動

```
2_editor — -bash — 80x24
Last login: Wed Apr 24 10:10:42 on ttys000
dh63-197:~ nibb$ cd ~/data/2_editor/
dh63-197:2_editor nibb$ pwd
/Users/nibb/data/2_editor
dh63-197:2_editor nibb$ emacs
```

```
2_editor — emacs — 80x24
Welcome to GNU Emacs, a part of the GNU operating system.

Type C-l to begin editing.

Get help          C-h (Hold down CTRL and press h)
Emacs manual      C-h r
Emacs tutorial    C-h t          Undo changes      C-x u
Buy manuals       C-h C-m        Exit Emacs       C-x C-c
Browse manuals    C-h i
Activate menubar  F10 or ESC ` or M-`
(`C-' means use the CTRL key. `M-' means use the Meta (or Alt) key.
If you have no Meta key, you may instead type ESC followed by the character.)

GNU Emacs 22.1.1 (mac-apple-darwin)
  of 2018-08-18 on osx337.sd.apple.com
Copyright (C) 2007 Free Software Foundation, Inc.

GNU Emacs comes with ABSOLUTELY NO WARRANTY; type C-h C-w for full details.
Emacs is Free Software--Free as in Freedom--so you can redistribute copies
of Emacs and modify it; type C-h C-c to see the conditions.
Type C-h C-d for information on getting the latest version.

----- GNU Emacs -----
For information about the GNU Project and its goals, type C-h C-p.
```

Emacsを扱う際の注意

- マウスは使えません。

メモ帳やテキストエディットとは異なり、
マウスを使用しようとしてもできません。
(カーソル移動や範囲選択などもできません)

→代わりにキーボードを使用します。

Emacs エディタのコマンド入力

- コントロール+文字キー
 - コントロールキーを押しながら文字キーを押す
 - C-X (Control+x) などと表記
- エスケープ+文字キー (メタキー)
 - Esc キーを押してから文字キーを押す
 - M-X (Meta+x) などと表記

Emacs の起動と終了

- 起動

\$ emacs [ファイル名]

(存在しないファイル名を指定すると、
その名前のファイルを作成する)

- 終了

- C-x C-c 保存するか聞かれるのでy (yes)またはn (no)と入力して終了
- C-z 編集を中断してシェルに戻る。fg で再開

演習)

- 1) \$ emacs と入力し、emacsを起動してください。
- 2) 起動したらemacsを終了してシェルに戻ってください。

ファイルの読み込みと保存

- ファイルの読み込み
 - C-x C-f で読み込むファイル名を入力
- ファイルの保存
 - C-x C-s 現在のファイル名で保存
 - C-x C-w 別名で保存

演習)

- 1) `$ emacs samtools_readme.txt` と入力し、emacsでファイルを読み込んでください。
- 2) 読み込んだら C-x C-w を使用し、このファイルを別名(README.txt)で保存してください。
- 3) 保存ができたらemacsを終了してシェルに戻り、
先ほど別名で保存したファイルがあることをlsコマンドで確認してください。

Emacs の基本的なコマンド

カーソル移動

- C-p(または↑)上に移動
- C-n(または↓)下に移動
- C-b(または←)左に移動
- C-f(または→)右に移動
- C-a 行の先頭に移動
- C-e 行の最後に移動
- C-v 1ページ分下に移動
- M-v 1ページ分上に移動
- M-< ファイルの先頭へ移動
- M-> ファイルの最後へ移動
- C-@ 現在の位置をマーク
- C-SPC 現在の位置をマーク
- C-x C-x カーソル位置との入れ替え

編集と検索

- DEL 左隣の文字を削除
- C-d カーソルの文字を削除
- C-k 現在の行のカーソル以降を削除してカットバッファへ
- C-w マーク位置からカーソル位置の間を削除してカットバッファへ
- C-y カットバッファの内容をペースト
- C-s 順方向に文字列検索
- C-r 逆方向に文字列検索
- C-_ 取り消し(Undo)
- C-g コマンド入力のキャンセル

実習)

README.txtをemacsで起動して
実際に操作してみましょう

カーソル移動

			M-< ファイル先頭へ			
			M-v 1ページ上へ			
			C-p(↑) 1つ上へ			
C-a 行の先頭へ		C-b(←) 1つ左へ		C-f(→) 1つ右へ		C-e 行の末尾へ
			C-n(↓) 1つ下へ			
			C-v 1ページ下へ			
			M-> ファイル最後へ			

編集

[DEL]	左隣の文字を削除
C-d	カーソル位置の文字を削除
C-@ または C- [SPACE]	現在の位置をマーク
C-w	マークした位置からカーソル位置までを削除してカットバッファへ (= カット)
M-w	マークした位置からカーソル位置までをコピーしてカットバッファへ (= コピー)
C-y	カットバッファの内容をペースト

検索・アンドゥ・キャンセル

C-s	順方向に文字列検索
C-r	逆方向に文字列検索
C-_ または C-x u	直前の編集を取り消し（アンドゥ）
C-g	コマンド入力のカンセル （例：C-sによる検索の中断）